Arkadeep Mondal 2405036 CS22 OPTION A

# Network Simulator Assignment

Prepared by Dr. Nitin Varyani

Packet routing in the same subnet and in different subnets. Use Cisco Packet Tracer (Option A) or NS-3 (Option B). The tasks and deliverables are the same for both.

## 1. About the tools and setup

**Cisco Packet Tracer** is a free visual simulator from Cisco Networking Academy: drag routers, switches and PCs onto a workspace, configure them, and watch packets in **Simulation mode**. **NS-3** is a free, open-source simulator used in research and teaching: you write a short C++ script, run it, and read the console output and packet captures (.pcap). **Key idea:** a host uses the subnet mask to decide if the destination is on its own subnet. If yes, it ARPs for the destination MAC and sends directly. If not, it sends the packet to its **default gateway** (the router), which forwards it using its **routing table**.

### Setup: Option A, Packet Tracer

1. Create a free account at **netacad.com** and enrol in **Getting Started with Cisco Packet Tracer**. 2. Download the installer for your OS, run it (keep defaults), then launch and sign in.

### Setup: Option B, NS-3 (Ubuntu/Linux; on Windows use WSL2 or an Ubuntu VM)

```
sudo apt update && sudo apt install g++ python3 cmake ninja-build git wireshark tcpdump wget
https://www.nsnam.org/releases/ns-allinone-3.45.tar.bz2 # or the current release tar xfj ns-allinone-3.45.tar.bz2 && cd ns-allinone-3.45/ns-3.45
./ns3 configure --enable-examples --enable-tests && ./ns3 build
./ns3 run first # test
```

## 2. Task 1: Routing of packets in a LAN within the same subnet

Four PCs in one subnet (no router). Same addresses in both options:

| Device | IP Address | Subnet Mask | Gateway |
|---|---|---|---|
| PC0 | 192.168.1.1 | 255.255.255.0 | - |
| PC1 | 192.168.1.2 | 255.255.255.0 | - |
| PC2 | 192.168.1.3 | 255.255.255.0 | - |
| PC3 | 192.168.1.4 | 255.255.255.0 | - |

| Step | Option A: Packet Tracer | Option B: NS-3 |
|---|---|---|
| 1. Build | Place 1 switch (2960) and 4 PCs. Connect PCs to Fa0/1 to Fa0/4 with straight-through cables. Wait for green links. | Save the Appendix A code as scratch/lab7-task1.cc (4 nodes on one LAN segment). |
| 2. Addresses | On each PC: **Desktop > IP Configuration**, enter IP and mask from the table. | Already set in the code. Check they match the table. |
| 3. Test connectivity | On PC0 (**Desktop > Command Prompt**): ping 192.168.1.2 then ping 192.168.1.4. | Run ./ns3 run scratch/lab7-task1. PC0 sends echo packets to PC1, then to PC3. |
| 4. Observe packets | Switch to **Simulation mode**, filter to ARP and ICMP, add a simple PDU from PC0 to PC3, press **Capture/Forward**. Note ARP first, then ICMP. | Run tcpdump -nn -e -tt -r lab7-task1-0-0.pcap. Note ARP request, ARP reply, then echo request and reply. |

| | | |
|---|---|---|
| 5. Check address tables | PC0: arp -a. Switch CLI: show mac address-table. | The ARP reply in the capture shows PC3's MAC (same information as arp -a). |
| 6. Experiment | On PC0 run ping 192.168.2.4 (a different network). Note the result. | Uncomment the 192.168.2.4 line in the code, rebuild, run. Note the result. |

## 3. Task 2: Routing of packets in a LAN with different subnets

Two subnets joined by router R1. PC0 and PC1 are on LAN A, PC2 and PC3 on LAN B. Same addresses in both options:

| Device | Interface | Address | Mask | Gateway |
|---|---|---|---|---|
| R1 | Gi0/0 (LAN A) | 192.168.10.1 | 255.255.255.0 | - |
| R1 | Gi0/1 (LAN B) | 192.168.20.1 | 255.255.255.0 | - |

| | | | | |
|---|---|---|---|---|
| PC0 | NIC | 192.168.10.2 | 255.255.255.0 | 192.168.10.1 |
| PC1 | NIC | 192.168.10.3 | 255.255.255.0 | 192.168.10.1 |
| PC2 | NIC | 192.168.20.2 | 255.255.255.0 | 192.168.20.1 |
| PC3 | NIC | 192.168.20.3 | 255.255.255.0 | 192.168.20.1 |

| Step | Option A: Packet Tracer | Option B: NS-3 |
|---|---|---|
| 1. Build | Place R1 (1941), 2 switches (2960), 4 PCs. PC0, PC1 to Switch0 to R1 Gi0/0. PC2, PC3 to Switch1 to R1 Gi0/1. | Save the Appendix B code as scratch/lab7-task2.cc (router node R1 joins LAN A and LAN B). |
| 2. Configure | R1 CLI: commands below. Each PC: IP, mask and **default gateway**. | Addresses are set in the code. Hosts get routes to the other LAN via R1 from PopulateRoutingTables(). |
| 3. Same subnet | PC0: ping 192.168.10.3 (PC1). | Run ./ns3 run scratch/lab7-task2. PC0 sends echo to PC1 at 2 s. |
| 4. Different subnet | PC0: ping 192.168.20.3 (PC3), then tracert 192.168.20.3. | PC0 sends echo to PC3 at 6 s (same run). |
| 5. Routing table | On R1: show ip route. | Read the routing tables printed at 1 s by the script. |
| 6. Packet at the router | Simulation mode: PDU from PC0 to PC3. Click it at PC0, R1 and PC3. Complete the table below. | Compare lab7-task2-0-0.pcap (R1, LAN A side) and lab7-task2-0-1.pcap (R1, LAN B side) with tcpdump -nn -e -r. Complete the table below. |
| 7. Fault test | R1: interface gi0/1, shutdown. Ping PC3 from PC0. Then no shutdown. | Uncomment the SetDown(2) line, rebuild, run. PC0 to PC3 echo now fails. Comment it out again. |

Router configuration (Option A, Router > CLI):

```
enable
configure terminal
hostname R1
interface gigabitEthernet 0/0
ip address 192.168.10.1 255.255.255.0
no shutdown
interface gigabitEthernet 0/1
ip address 192.168.20.1 255.255.255.0
no shutdown
end
```

**Observation table (PC0 to PC3, both options)**

| Field | On LAN A (PC0 to R1) | On LAN B (R1 to PC3) | Changed or Same |
|---|---|---|---|
| Source IP | 192.168.10.2 | 192.168.10.2 | same |
| Destination IP | 192.168.20.2 | 192.168.20.2 | same |
| Source MAC | 0060.4755.7839 | 000c.cf2b.9201 | changed |
| Destination MAC | 000c.cf2b.9201 | 00D0.584E.627A | changed |

## 4. Deliverables (same for both options)

| Item | Packet Tracer | Option B: NS-3 |
|---|---|---|
| Topology and addressing | Screenshot of each workspace, labelled, plus completed addressing tables. | Neat diagram of each topology (nodes, links, IPs) plus completed addressing tables. |
| Connectivity proof | Ping screenshots (Task 1: PC1 and PC3; Task 2: same and different subnet) and the tracert output. | Console output screenshots showing "client sent" and "client received" for each echo. |
| Packet observation | Simulation mode screenshots (ARP, then ICMP) for Task 1 and Task 2. | tcpdump output screenshots (ARP, then echo) for Task 1 and Task 2. |
| Tables | Task 1: arp -a and MAC address table. Task 2: show ip route. | Task 1: ARP reply MAC. Task 2: printed routing tables. |
| Observation table | Completed table from Task 2. | Completed table from Task 2. |
| Experiment and fault test | Screenshots and result of ping 192.168.2.4 and of the Gi0/1 shutdown. | Console output of the 192.168.2.4 experiment and of the SetDown fault test. |
| Files | Lab7_Task1.pkt and Lab7_Task2.pkt | lab7-task1.cc and lab7-task2.cc |

Arkadeep Mondal 2405036 CS22 OPTION A

## Short written answers (same for both)

### 1. Why is the first ping/echo slower than the later ones?

The first ping requires address resolution via ARP to discover the destination's (or gateway's) MAC address before ICMP packets can be sent. Subsequent pings use the cached MAC address directly, making them faster.

### 2. Is a router involved in Task 1? Explain.

No. All devices are on the same subnet (192.168.1.0/24) and communicate directly through the Layer 2 switch using MAC addresses resolved by ARP.

### 3. What happened when PC0 sent to 192.168.2.4, and why?

The ping failed. Since 192.168.2.4 is in a different subnet, PC0 attempted to forward the packet to its default gateway, but no default gateway or router was configured in Task 1.

### 4. Which address fields changed as the packet crossed R1, and which stayed the same?

The IP addresses stayed the same for end-to-end routing. The MAC addresses changed as R1 rewrote the Layer 2 Ethernet frame header for LAN B

### 5. Why does a PC need a default gateway? What did the fault test show?

A PC needs a default gateway to forward traffic addressed to IP subnets outside its local LAN. The fault test showed that shutting down the gateway interface (gi0/1) breaks inter-subnet connectivity, causing pings between subnets to fail.

## How to submit

Submit **one ZIP file** named RollNo_Name_Lab7.zip containing: (1) Lab7_Report.pdf (or .docx) with your name, roll number, option used, screenshots, tables and written answers; and (2) your simulator files: Lab7_Task1.pkt and Lab7_Task2.pkt (Option A) or lab7-task1.cc and lab7-task2.cc (Option B).

## Appendix A (Option B): scratch/lab7-task1.cc

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/csma-module.h"
#include "ns3/internet-module.h"
#include "ns3/applications-module.h"
using namespace ns3;

// Sends 2 UDP echo packets from one PC to another (works like a 2-packet ping)
static void Echo(Ptr<Node> from, Ipv4Address to, double start) {
 UdpEchoClientHelper c(to, 9);
 c.SetAttribute("MaxPackets", UintegerValue(2));
 c.SetAttribute("Interval", TimeValue(Seconds(1)));
 c.SetAttribute("PacketSize", UintegerValue(512));
 ApplicationContainer a = c.Install(from);
 a.Start(Seconds(start)); a.Stop(Seconds(start + 3));
}

int main() {
 LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_INFO);
 LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_INFO);
 CsmaHelper csma;
 csma.SetChannelAttribute("DataRate", StringValue("100Mbps"));
 csma.SetChannelAttribute("Delay", TimeValue(NanoSeconds(6560)));
 NodeContainer pc; pc.Create(4); // PC0..PC3
 NetDeviceContainer dev = csma.Install(pc); // one LAN segment
 InternetStackHelper stack; stack.Install(pc);
 Ipv4AddressHelper ip; ip.SetBase("192.168.1.0", "255.255.255.0");
 Ipv4InterfaceContainer ifc = ip.Assign(dev); // PC0=.1 PC1=.2 PC2=.3 PC3=.4  UdpEchoServerHelper server(9); // echo servers
on PC1 and PC3  ApplicationContainer sa = server.Install(NodeContainer(pc.Get(1), pc.Get(3)));  sa.Start(Seconds(1));
sa.Stop(Seconds(15));
 Echo(pc.Get(0), ifc.GetAddress(1), 2); // PC0 -> PC1
 Echo(pc.Get(0), ifc.GetAddress(3), 6); // PC0 -> PC3
 // Echo(pc.Get(0), Ipv4Address("192.168.2.4"), 10); // EXPERIMENT: uncomment  csma.EnablePcapAll("lab7-task1",
true); // packet capture files  Simulator::Run(); Simulator::Destroy();
 return 0;
 }
```

## Appendix B (Option B): scratch/lab7-task2.cc

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/csma-module.h"
#include "ns3/internet-module.h"
#include "ns3/applications-module.h"
using namespace ns3;
```

# Arkadeep Mondal 2405036 CS22 OPTION A

```
// Sends 2 UDP echo packets from one PC to another (works like a 2-packet ping) static void Echo(Ptr<Node> from,
Ipv4Address to, double start) {
 UdpEchoClientHelper c(to, 9);
 c.SetAttribute("MaxPackets", UintegerValue(2));
 c.SetAttribute("Interval", TimeValue(Seconds(1)));
 c.SetAttribute("PacketSize", UintegerValue(512));
 ApplicationContainer a = c.Install(from);
 a.Start(Seconds(start)); a.Stop(Seconds(start + 3));
}

int main() {
 LogComponentEnable("UdpEchoClientApplication", LOG_LEVEL_INFO);
 LogComponentEnable("UdpEchoServerApplication", LOG_LEVEL_INFO);
 CsmaHelper csma;
 csma.SetChannelAttribute("DataRate", StringValue("100Mbps"));
 csma.SetChannelAttribute("Delay", TimeValue(NanoSeconds(6560)));
 NodeContainer router, lanA, lanB;
 router.Create(1); lanA.Create(2); lanB.Create(2); // R1 | PC0,PC1 | PC2,PC3  NetDeviceContainer dA =
csma.Install(NodeContainer(router, lanA));
 NetDeviceContainer dB = csma.Install(NodeContainer(router, lanB));
 InternetStackHelper stack;
 stack.Install(router); stack.Install(lanA); stack.Install(lanB);
 Ipv4AddressHelper ip;
 ip.SetBase("192.168.10.0", "255.255.255.0");
 Ipv4InterfaceContainer iA = ip.Assign(dA); // R1 .1 PC0 .2 PC1 .3
 ip.SetBase("192.168.20.0", "255.255.255.0");
 Ipv4InterfaceContainer iB = ip.Assign(dB); // R1 .1 PC2 .2 PC3 .3
 Ipv4GlobalRoutingHelper::PopulateRoutingTables(); // build routing tables  Ptr<OutputStreamWrapper> rt =
Create<OutputStreamWrapper>(&std::cout);  Ipv4GlobalRoutingHelper g; g.PrintRoutingTableAllAt(Seconds(1), rt);  //
FAULT TEST: uncomment to shut down R1's LAN B interface
 // router.Get(0)->GetObject<Ipv4>()->SetDown(2);
 UdpEchoServerHelper server(9); // echo servers on PC1 and PC3  ApplicationContainer sa =
server.Install(NodeContainer(lanA.Get(1), lanB.Get(1)));  sa.Start(Seconds(1)); sa.Stop(Seconds(15));
 Echo(lanA.Get(0), iA.GetAddress(2), 2); // PC0 -> PC1 (same subnet)  Echo(lanA.Get(0), iB.GetAddress(2), 6); // PC0 -> PC3
(different subnet)  csma.EnablePcapAll("lab7-task2", true);
 Simulator::Run(); Simulator::Destroy();
 return 0;
}
```